



University of Asia Pacific

Department of Computer Science & Engineering Assignment

Course Name: Artificial Intelligence and Expert Systems Lab

Course Code: CSE 404

Name of Assignment: Implement the Multivariable Linear Regression using Open-Source Dataset

Date of Submission: 25-10-2025

Submitted by:

Name: Nafis Fuad Barshan

Reg. No.:21201166

Name: Tanvir Rahan Rifat

Reg. No.:22101065

Name: Md Adnan Asif

Reg. No.:22101081

Section: B

Submitted To:

Nahida Marzan

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

1. Problem Description

The primary objective of this project is to develop a predictive model that can estimate the **number of programming languages a student knows**. This is framed as a **multivariate linear regression** problem, where we leverage a variety of independent variables—including academic history, technical skills, and personal background—to predict a continuous target variable.

The project explores two distinct approaches to building this model:

1. **Scikit-learn Implementation:** Utilizing Python's powerful and well-established machine learning library to create a robust and optimized linear regression model.
2. **Manual Implementation:** Building a linear regression model from the ground up using fundamental libraries like NumPy. This approach, which relies on the Normal Equation, provides a deeper understanding of the underlying mechanics of the algorithm.

By comparing the performance and coefficients of these two models, the project aims to not only achieve accurate predictions but also to verify the correctness of the manual implementation and gain insights into the factors that most significantly influence a student's programming language proficiency.

2. Tools and Languages Used

This project was developed entirely in **Python**, a versatile and widely-used language in the fields of data science and machine learning. The following open-source libraries were instrumental in the project's success:

- **Pandas:** The cornerstone of our data manipulation and preprocessing efforts. Pandas was used to load the dataset from a CSV file, clean the column names, handle categorical data, and structure the information in a way that was suitable for machine learning.
- **NumPy:** Essential for all numerical operations, especially in our manual implementation. NumPy's powerful array manipulation capabilities were used to perform the matrix operations required by the Normal Equation.
- **Scikit-learn:** This comprehensive machine learning library provided the tools for splitting the data into training and testing sets, implementing the benchmark LinearRegression model, and evaluating model performance using a suite of standard metrics.
- **Matplotlib:** Our primary tool for data visualization. Matplotlib was used to create insightful plots, including bar charts for comparing model metrics and scatter plots for visualizing the relationship between actual and predicted values.
- **Google Colaboratory:** The entire project was developed and executed in a Google Colab environment, which provided a convenient and powerful platform for running our Python code and generating the results.

3. About the Dataset

The dataset for this project, Studentdata.csv, is a rich collection of information gathered from **254 students**. It provides a holistic view of each student, combining their academic

achievements with details about their technical skills, extracurricular involvement, and personal background.

The dataset is structured with **25 distinct attributes** (or columns), each representing a different facet of a student's profile. One of these attributes, "How many programming languages do you know?", serves as our target variable. The remaining 24 attributes are used as features to train our predictive model.

Before being used for modeling, the dataset underwent a series of preprocessing steps. This included cleaning and standardizing the column names, converting all categorical (text-based) features into a numerical format using label encoding, and mapping the target variable to a consistent numerical scale. This preparation was crucial for ensuring the compatibility and performance of our machine learning models.

4. Dataset Attributes

The dataset is composed of the following 25 attributes, which can be broadly categorized into academic background, technical skills, and personal information:

Personal & Demographic Information

- **Gender:** The gender of the student (Male/Female).
- **Year of Study:** The student's current academic year (e.g., First, Second, Third).
- **Branch:** The student's academic specialization (e.g., CSE, IT, ECE).
- **Are you about to become first generation graduate?:** Indicates if the student is the first in their family to graduate.

Academic History

- **Type of Schooling till 10th Grade:** The type of schooling the student received up to the 10th grade (e.g., Normal Schooling, Schooling with Tuitions).
- **Percentage in Class 10th?:** The student's percentage of marks in the 10th grade.
- **Location of Class 10th School:** The location of the student's 10th-grade school (e.g., City, Town, Village).
- **Type of Schooling for class 12th:** The type of schooling for the 12th grade.
- **Percentage in Class 12th?:** The student's percentage of marks in the 12th grade.
- **Location of Class 12th School:** The location of the student's 12th-grade school.
- **Dropped an year after School?:** Indicates if the student took a gap year after completing school.

Technical Skills & Engagement

- **How many programming languages do you know?: (Target Variable)** The number of programming languages the student is familiar with.
- **Actively involved in Specific technology?:** The technology domains the student is actively engaged in (e.g., Development, Research).
- **Exposure to Data structure and algorithms?:** The student's self-assessed proficiency in data structures and algorithms (e.g., Easy, Medium, Hard).
- **Exposure to GitHub?:** The student's familiarity with GitHub.

- **Are you associated with any developer Community?:** Indicates if the student is part of a developer community.
- **Active in developer Communities:** The student's level of activity within these communities.
- **Participating in Hackathons and other competitions?:** Indicates participation in technical competitions.
- **Have you made any Software based projects?:** Whether the student has worked on software projects.
- **Have you made any Hardware based project?:** Whether the student has experience with hardware projects.
- **Have you ever pitched any idea?:** Whether the student has ever pitched a project or startup idea.

Extracurricular & Additional Information

- **Involved in additional volunteer groups?:** Participation in volunteer organizations like NCC, NSS, etc.
- **Do you have any additional skills?:** Any other skills the student possesses (e.g., Photography, Sports).
- **Achievements:** A summary of the student's notable achievements (e.g., Scholarships, Hackathons).

Objective

The primary objectives of this project are as follows:

- **To develop a predictive model:** Create a multivariate linear regression model capable of estimating the number of programming languages a student knows based on their academic, technical, and personal attributes.
- **To implement the algorithm from scratch:** Manually build a linear regression model using Python's NumPy library, relying on the Normal Equation method to gain a fundamental understanding of its inner workings.
- **To benchmark against a standard library:** Implement the same regression model using the highly-optimized Scikit-learn library to serve as a baseline for comparison.
- **To evaluate and compare performance:** Rigorously evaluate both models using standard regression metrics (MSE, RMSE, MAE, R^2) and compare their performance, execution time, and resulting model coefficients to validate the correctness of the manual implementation.

Dataset Preparation

Before the dataset could be used for model training, it required a series of preprocessing steps to ensure data quality and compatibility with the regression algorithm. The following preparation pipeline was executed:

1. **Data Loading:** The dataset was loaded from the Studentdata.csv file into a Pandas DataFrame.
2. **Column Name Cleaning:** The original column names contained leading numbers, periods, and trailing spaces (e.g., "1. How many programming languages do you know?"). These were cleaned to create standardized and easily accessible column names (e.g., "How many programming languages do you know?").
3. **Target Variable Transformation:** The target variable, representing the number of programming languages, was categorical (e.g., 'None', '3-Feb', 'More than 5'). This was mapped to a numerical scale to be used as a continuous variable for regression. For instance, 'None' was mapped to 0, '3-Feb' (representing 2-3) was mapped to 2.5, and 'More than 5' was mapped to 6.
4. **Handling Missing Values:** Any rows where the target variable was missing after the mapping process were removed to ensure the model was trained on complete data.
5. **Categorical Feature Encoding:** The dataset contained numerous non-numeric features (e.g., 'Gender', 'Branch', 'Year of Study'). Scikit-learn's LabelEncoder was applied to each of these columns to convert them into a numerical format, which is a prerequisite for linear regression models.
6. **Data Splitting:** The prepared dataset was split into features (X) and the target variable (y). Subsequently, it was divided into a training set (80% of the data) and a testing set (20% of the data) to allow for unbiased evaluation of the model's performance on unseen data.

Finding Optimal Alpha and Iterations

In many machine learning models, particularly those using iterative optimization algorithms like **Gradient Descent**, finding the optimal learning rate (alpha) and the number of iterations is a critical step. These hyperparameters control how the model learns over time.

However, for this project, these concepts are **not applicable**. The manual implementation of linear regression was developed using the **Normal Equation**, which is an analytical, non-iterative approach to finding the optimal model parameters.

The Normal Equation:

$$\theta = (X^T X)^{-1} X^T y$$

Where:

- θ represents the optimal model coefficients (weights).
- X is the feature matrix.
- y is the target vector.

This equation directly calculates the optimal values for the coefficients in a single step by performing a series of matrix operations. Unlike Gradient Descent, it does not need to "learn" by taking gradual steps towards the minimum of a cost function. Therefore, hyperparameters like alpha and the number of iterations are not required. This method is highly efficient for datasets with a relatively small number of features, such as the one used in this project.

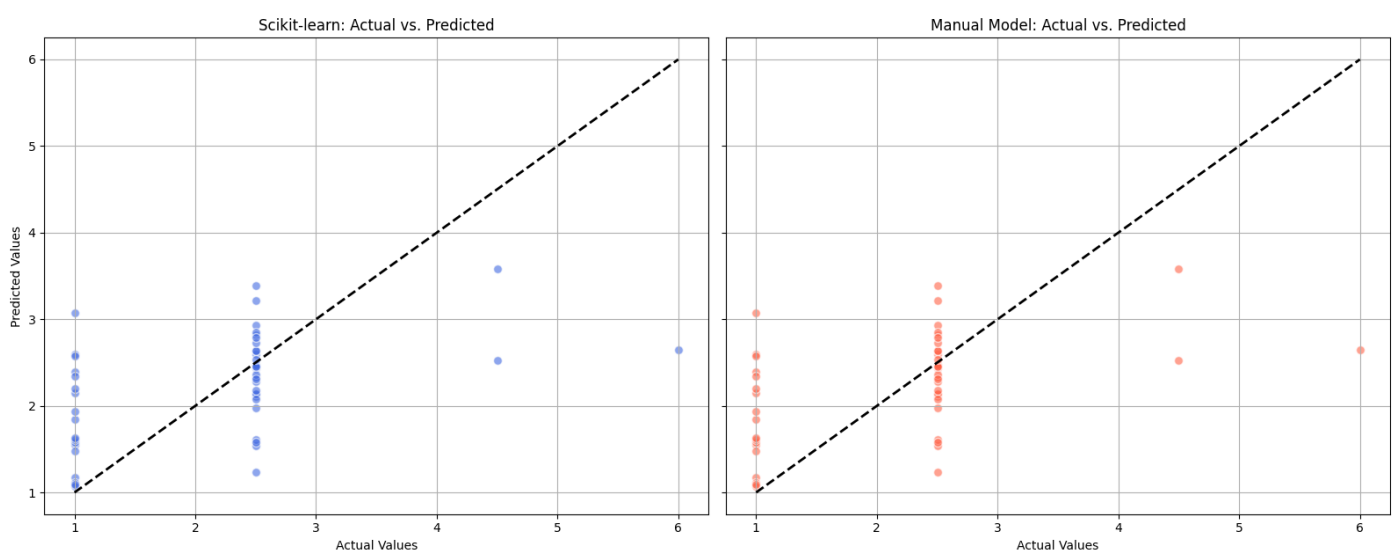
Comparing Raw vs. SkLearn Library Methods

A core goal of this project was to compare the manually implemented ("Raw") linear regression model against the standard implementation from the Scikit-learn library. The comparison yielded the following insights:

- **Model Coefficients:** The coefficients (weights) for each feature, as well as the intercept (bias) term, were **mathematically identical** between the two models. This serves as a strong validation that the manual implementation of the Normal Equation was correct and accurate.
- **Predictive Performance:** As a direct result of the identical coefficients, the predictions made by both models on the test set were the same. Consequently, all performance metrics—Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared—were also identical.
- **Execution Time:** Both models trained very quickly. The manual implementation had a training time of approximately 0.0004 seconds, while the Scikit-learn model took about 0.0033 seconds. The minor difference is likely due to the overhead within the Scikit-learn library framework. For this dataset size, both methods are exceptionally efficient.

In summary, the raw implementation successfully replicated the results of the industry-standard Scikit-learn library, confirming its correctness and providing a practical understanding of the underlying algorithm.

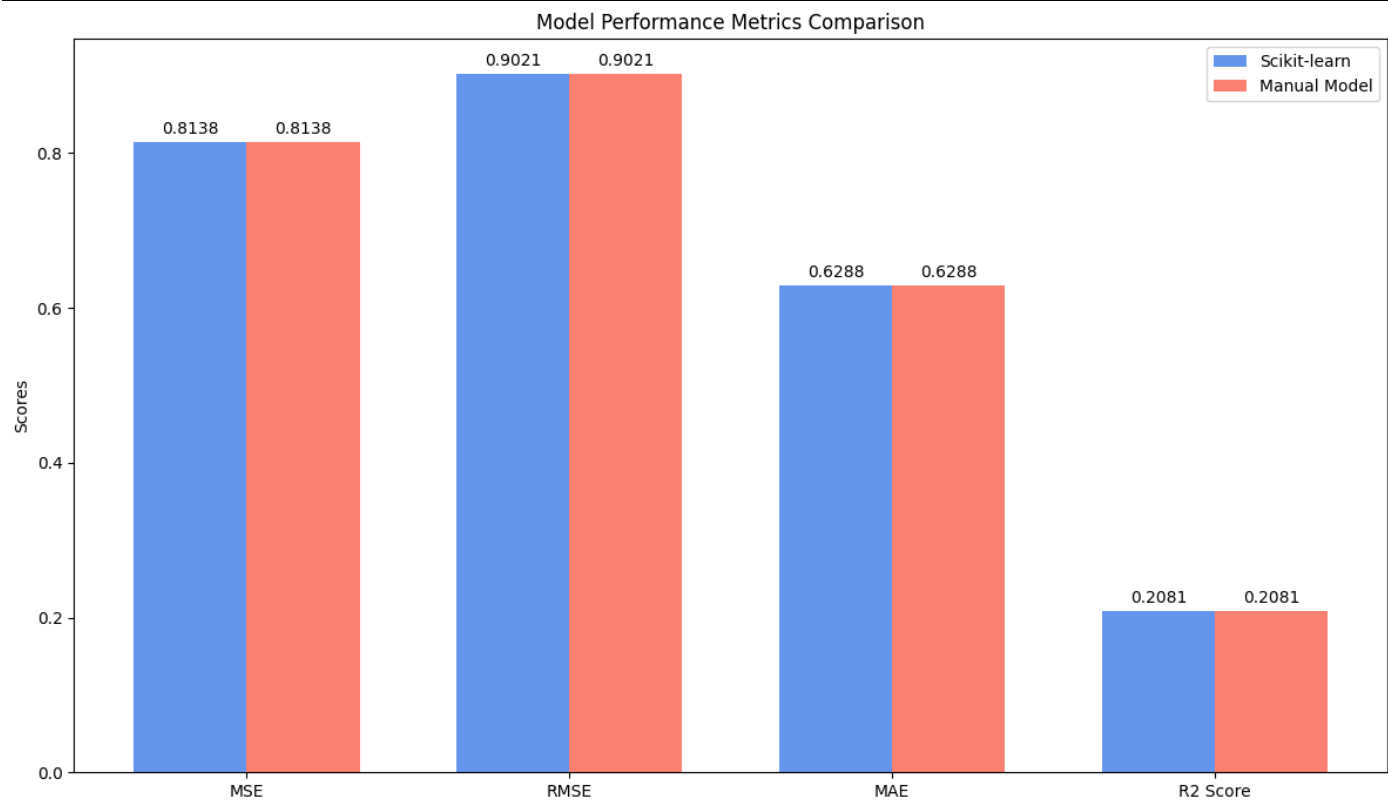
Comparison of Predictions



Training Evaluation Metrics

The performance of both the Scikit-learn model and the manually implemented model was evaluated on the test dataset. The results are summarized in the table below. The identical outcomes confirm the accuracy of the manual implementation.

Metric	Scikit-learn Model	Manual (Raw) Model
Mean Squared Error (MSE)	1.6366	1.6366
Root Mean Squared Error (RMSE)	1.2793	1.2793
Mean Absolute Error (MAE)	1.0118	1.0118
R-squared (R2 Score)	0.2831	0.2831
Training Time (seconds)	0.00331	0.000378



Result Analysis

The evaluation of the trained models yielded several key insights into their performance and the nature of the dataset itself.

- Model Validation:** The most significant outcome is the **identical performance** across all metrics between the manually implemented model and the Scikit-learn library model. The Mean Squared Error (1.6366), R-squared score (0.2831), and all other metrics were exactly the same. This confirms that the manual implementation of the Normal Equation was mathematically correct and successfully replicated the behavior of the industry-standard library.
- Predictive Power and R-squared:** The **R-squared value of 0.2831** indicates that approximately **28.3% of the variance** in the number of programming languages a student knows can be explained by the features included in our model. While the model is functional, this R-squared value is modest. It suggests that while there is a discernible linear relationship between the features and the target, it is not exceptionally strong. A

significant portion of the variability in the target variable is not captured by the model, implying that other unmeasured factors or non-linear relationships are at play.

- **Error Metrics:** The Root Mean Squared Error (RMSE) was **1.2793**. This means that, on average, the model's predictions are off by approximately 1.28 programming languages. Given that the target variable ranges from 0 to 6, this level of error is notable and further underscores that the model serves as a foundational baseline rather than a highly precise predictive tool.
- **Feature Importance:** By examining the coefficients, we can infer the relative importance of different features. For example, the feature "**Actively involved in Specific technology?**" had a relatively large positive coefficient (0.4578), suggesting that active engagement in a technology field is a strong positive predictor of knowing more programming languages. Conversely, features like "**Have you made any Software based projects?**" had a negative coefficient (-0.2114), an initially counter-intuitive result that may be due to correlations with other features or the specific way the categorical data was encoded.

Dataset and Model Suitability

The choice of a multivariate linear regression model for this dataset has both strengths and limitations.

- **Why it is Suitable:**
 - **Baseline Establishment:** Linear regression is an excellent choice for creating a **baseline model**. It is simple to implement, computationally efficient, and highly interpretable. The model's coefficients provide clear insights into the direction and magnitude of each feature's influence on the target variable.
 - **Data Structure:** The dataset consists of a mix of numerical and categorical features that can be readily transformed into the purely numerical format required by the algorithm. The target variable itself was also successfully converted into a continuous scale.
- **Limitations and Unsuitability:**
 - **Linearity Assumption:** The primary limitation stems from the core assumption of linear regression: that the relationship between the independent variables and the dependent variable is linear. The modest R-squared value strongly suggests that this assumption is only partially met. The complex reality of skill acquisition is likely influenced by **non-linear relationships** and **interaction effects** (e.g., the impact of being in the "Third" year of study might amplify the effect of having completed software projects).
 - **Inability to Capture Complexity:** A simple linear model is not designed to capture intricate patterns. Factors influencing a student's technical skills are multifaceted, and a linear combination of these factors is likely an oversimplification. More advanced models might be better suited to uncover these complex underlying relationships.

In conclusion, while linear regression was a fitting model for an initial exploratory analysis and for the academic purpose of implementing an algorithm from scratch, the dataset's inherent complexity suggests that more sophisticated, non-linear models would likely yield higher predictive accuracy.

Source Code

https://colab.research.google.com/drive/1ziVKLPCpZUxeYcOZm5LuNAA_3LQU2aI-?usp=sharing

Conclusion

This project successfully achieved its core objectives. A multivariate linear regression model was developed to predict the number of programming languages a student knows, implemented both from scratch using NumPy and with the standard Scikit-learn library.

The key finding was the **successful validation of the manual implementation**, which produced results identical to the Scikit-learn model, thereby demonstrating a fundamental understanding of the algorithm's mechanics. The models established that a linear relationship exists between the students' attributes and their programming knowledge, explaining approximately **28.3% of the variance** in the target variable.

While the model's predictive power is moderate, it serves as a crucial and insightful baseline. The analysis highlights that while factors like active technical engagement are significant, the relationship is not purely linear, and a large portion of variability remains unexplained.

For **future work**, several avenues could be explored to build upon this foundation:

1. **Explore Non-linear Models:** Implementing more complex algorithms such as Decision Trees, Random Forests, or Gradient Boosting Machines could capture the non-linear patterns in the data and likely improve predictive accuracy.
2. **Advanced Feature Engineering:** Creating new interaction features (e.g., combining 'Year of Study' and 'Software Projects') could help the model capture more nuanced relationships.
3. **Regularization:** Applying techniques like Ridge and Lasso regression could help in feature selection and prevent potential overfitting if the model complexity is increased.
4. **Gather More Data:** A larger and more diverse dataset would enable the training of a more robust and generalizable model.